Intern Challenge: The 3-Day Build

One problem, three days, one working system.

Part 1 is for candidates. Part 2 (Interviewer Guide) is internal and must be removed before sending.

PART 1: CANDIDATE BRIEF

1. Welcome

Thanks for taking the time to do this. We are a startup. That means small team, real users, incomplete information, and things that break on a Tuesday afternoon. This assignment is built to feel like a compressed version of working with us.

You will choose **one of two projects**. Both run for **three days**. Each day builds directly on the previous day, and by the end of Day 3 you must deliver one complete, working, explainable system, not three disconnected exercises.

There is no single correct answer. We care about **how you think, how you decide, how you recover when things go wrong, and how clearly you communicate.** A modest solution that is honest, well-tested, and well-explained beats an impressive one you cannot defend.

2. Choose your project

Read both before choosing. You must pick exactly one. Send us one line telling us which and why (this counts toward your evaluation).

	Project Parcel	Project Concierge
One-line pitch	Predict which shipments will arrive late, and tell operations exactly what to do about it	Build a support assistant that answers customer questions correctly from messy company knowledge, and knows when it should not answer
You will mostly work with	Tables of historical data with real-world flaws	Documents, text, and a small set of tools and live-style services

	Project Parcel	Project Concierge
The hard part	Not fooling yourself about whether your predictions are actually good	Making a system you can trust when its outputs are open-ended and unpredictable
Difficulty	Advanced	Advanced

Both are hard on purpose. In previous rounds, only a small fraction of people finish every task. **Finishing everything is not the goal; making good decisions about what to finish is.**

3. Ground rules (apply to both projects)

1. **Time box.** Plan for roughly 5 to 6 focused hours per day. Do not work 14 hours. We would rather see good prioritization than exhaustion. Track your real hours.
2. **Tools.** You may use any language, library, editor, or coding assistant. If you use an assistant, you must disclose where and how. We will ask you to explain any line of your work, so use help you can defend.
3. **Questions are allowed and encouraged.** Ask in the shared channel. Some details are intentionally missing. Asking a sharp question is a positive signal. Asking a question whose answer is in the brief is not.
4. **Version control.** Use a Git repository with regular, meaningful commits. We will read your history.
5. **Daily check-in (mandatory).** By end of each day, send a short note using the template in Section 7. Missing a check-in without notice counts against you more than a weak result does.
6. **Failure and Recovery Log (mandatory).** Keep a running file named `FAILURES.md`. Every time something breaks, a result surprises you, or you abandon an approach, log it: what happened, what you believed, what you tried, what you learned. This file is one of the most important things we read.
7. **Surprises.** At the start of Day 2 and Day 3 we will send you an update (new data, changed requirements, a broken dependency). This simulates real life. How you react matters more than whether you get it perfect.
8. **Be honest.** If something does not work, say so plainly and show what you know about why. Never hide a weak result. Inflated claims are the fastest way to fail this assignment.

4. Final deliverables (both projects)

- A Git repository that runs from a clean checkout using a README with exact steps
- A short report (maximum 4 pages) following the structure in Section 8
- A recorded or live walkthrough of 5 to 8 minutes covering what you built, what failed, and what you would do next
- `FAILURES.md` and three daily check-in notes
- A 30-minute live review with us after submission, where we ask you to explain and modify parts of your work

---

5. PROJECT PARCEL

Context

A logistics startup ships thousands of parcels daily through several carriers and warehouses. Customers are complaining about late deliveries. The operations head, who is not technical, wants to know in advance which parcels are likely to be late so her team can intervene (reroute, upgrade shipping, or message the customer) before it happens. Interventions cost money, so she cannot flag everything.

You will receive historical shipment records, a carrier reference table, and a warehouse capacity file. Treat the data as you would data from a real company: it has flaws, and nobody has documented them for you.

Day 1: Understand it before you touch it

Goal: Prove you understand the problem and the data well enough to trust any model built on it.

Tasks:

1. **Define the problem precisely.** What exactly counts as late? Late compared to what promise? Write the definition and justify it. Different reasonable definitions will give you different labels. Choose one and defend it.
2. **Audit the data.** Find and document every quality issue you can: missing values, duplicates, impossible values, inconsistent formats, timezone problems, unit mismatches, and anything that looks suspicious. For each issue, state what you decided to do and why.

3. **Decide what you are allowed to use.** For every column, decide whether its value would truly be known *at the moment you need to make the prediction*. Some columns are only known after the outcome. Document your reasoning column by column.
4. **Build a deliberately simple baseline** (for example a rule or a basic model) with a proper evaluation. This number is your anchor for the next two days.
5. **Plan.** Write a one-page plan for Days 2 and 3 with your top three risks.

End of Day 1 output: audit notes, baseline result, the plan, check-in note.

Day 2: Build something that survives reality

Goal: Improve on your baseline in a way that you can show is real, not accidental.

At the start of Day 2 we will send an update to the data. Expect changes you were not told about.

Tasks:

1. **Handle the update.** Integrate the new data. Identify what changed compared to Day 1, explain how it affects your earlier work, and fix whatever it broke.
2. **Build your main approach.** Use features, models, and techniques of your choice. Compare at least two meaningfully different approaches, not just parameter tweaks of the same one.
3. **Validate honestly.** Choose an evaluation strategy that mimics how the system will be used in the future. Explain why a random split might be misleading here, and what you did instead.
4. **Pick the metrics that matter to the business.** Operations can only act on a limited number of parcels per day, and a missed late parcel costs more than a false alarm. Choose your metrics and your decision threshold, and justify them with a simple cost argument. Make the cost assumptions explicit.
5. **Break your own model.** Find the segments (carrier, region, season, parcel type) where it performs worst. Show evidence, not guesses.

End of Day 2 output: updated repository, comparison of approaches, error analysis, updated `FAILURES.md`, check-in note.

Day 3: Make it usable and trustworthy

Goal: Turn your analysis into something a team could actually run and rely on.

At the start of Day 3 we will send a final update: an operational constraint and a request that changes your priorities.

Tasks:

1. **Package it.** Provide a simple way to get predictions for new parcels (a script, command line, or small service). It must validate its inputs and fail clearly on bad data instead of silently producing nonsense.
2. **Explain it.** For any single flagged parcel, show a plain-language reason the system flagged it. Also explain what the system as a whole relies on most, and whether you trust that.
3. **Plan for decay.** Describe how you would know, a few months later, that the system has quietly stopped working. Implement at least one concrete check.
4. **Test it.** Add automated tests for the parts that matter most. Explain what you chose not to test and why.
5. **Write the decision memo** for the operations head: what to do, expected benefit, expected cost, what could go wrong, and what you are unsure about. No jargon.

End of Day 3 output: final repository, report, walkthrough, `FAILURES.md`, check-in note.

---

6. PROJECT CONCIERGE

Context

A consumer startup receives hundreds of support questions daily about orders, refunds, delivery, and account issues. The answers live in a messy pile of company documents: help articles, policy pages, internal notes, and old announcements. Some of these contradict each other, some are outdated, and some questions cannot be answered from them at all. The founder wants an assistant that gives correct, sourced answers, takes safe actions through company tools, and refuses or escalates when it should.

You will receive the document collection, a mock order service (a small API that is slow and occasionally fails), and a handful of sample customer messages. You may use any hosted or open language model. We will provide a usage budget; respect it.

Day 1: Knowledge and measurement first

Goal: Build a trustworthy way to find the right information, and a way to know if you are right.

Tasks:

1. **Study the documents.** Document the contradictions, outdated content, duplicates, and gaps you find. Decide how your system should treat each kind of problem.
2. **Build retrieval.** Make the documents searchable so the right passages can be found for a question. Decide how to split documents and justify it.
3. **Create your own evaluation set before building more.** Write at least 40 realistic questions with the correct answer and the source for each. Include questions that are ambiguous, questions with no answer in the documents, questions that need two documents combined, and questions that try to trick the system.
4. **Measure retrieval alone.** Report how often the right passage is actually found, and look at the failures.
5. **Plan** Days 2 and 3 with your top three risks.

End of Day 1 output: document issues list, working retrieval, evaluation set, baseline numbers, plan, check-in note.

Day 2: Answers, actions, and adversaries

Goal: Produce answers you can trust and handle things going wrong.

At the start of Day 2 we will update one of the policy documents and change the behavior of the order service.

Tasks:

1. **Handle the update.** Make your system reflect the new policy without a full rebuild. Show that old, wrong answers no longer appear. Adapt to the changed service.
2. **Generate answers with sources.** Every claim should point to its supporting passage. Design the system so that unsupported claims are detected or avoided. Define exactly what your system does when it does not know.
3. **Use the tools.** Let the system look up an order and use that information in its answer. Handle slow responses, errors, and malformed output from the service without crashing or inventing data.
4. **Attack it.** Write at least 10 adversarial inputs: attempts to make it ignore its instructions, reveal internal notes, promise refunds it should not, or act on another customer's order. Show how it responds and fix what fails.
5. **Score it.** Run your evaluation set and report results. Explain how you decided whether an answer was correct, since this cannot be a simple string match.

End of Day 2 output: updated system, scored results, attack log, updated `FAILURES.md`, check-in note.

Day 3: Reliability, cost, and judgment

Goal: Make it something a small team could responsibly put in front of customers.

At the start of Day 3 we will send a new constraint on cost and response time.

Tasks:

1. **Meet the constraint.** Reduce cost or latency to the new targets. Show what you gave up in quality, with measured numbers, not impressions.
2. **Compare two designs.** Evaluate two meaningfully different configurations (for example different retrieval strategies, model sizes, or prompting approaches) on the same evaluation set and recommend one.
3. **Know your limits.** Define when the system must hand over to a human. Implement it and test it, including on emotional or high-stakes messages.
4. **Protect against regressions.** Create an automated check that re-runs your evaluation set so that future changes cannot silently make things worse.
5. **Write the decision memo** for the founder: should we launch, to whom, with which safeguards, and what would you watch in the first month. Be honest about risk.

End of Day 3 output: final repository, report, walkthrough, `FAILURES.md`, check-in note.

---

7. Daily check-in template

Send this by end of each day (maximum 10 lines):

1. What I finished today
2. What I did not finish, and why
3. The biggest problem I hit and how I handled it
4. A decision I made that I am unsure about
5. Where I need input, if anywhere
6. Hours actually spent
7. Plan for tomorrow

8. Report structure (maximum 4 pages)

1. Problem as I understood it, including assumptions
2. Approach and key decisions, with alternatives I rejected and why
3. Results, with honest uncertainty
4. What failed and what I learned (summary of `FAILURES.md`)
5. Risks, limitations, and what I would do with two more weeks
6. How to run everything

9. How we will evaluate you (what you can see)

We look at your thinking more than your score. In rough order of importance: how well you frame the problem, how honestly you evaluate your own work, how you respond to surprises and failures, technical depth and correctness, quality and clarity of your engineering, how you communicate, and how reliably you show up and follow through. We will not reward complexity for its own sake.
